

Relatório Técnico — Depuração de Ativos EUA / Alpha Vantage / Cálculo de Quantidade

Gerado para análise técnica e exportação em PDF.

1) Onde a atualização de cotação dos EUA está sendo executada

O fluxo real do projeto atual é:

1. **App dispara o refresh** em `src/App.jsx`, onde a prop `onUpdateQuote` chama `handleUpdateAsset(asset, t, onSuccess)`.
2. **Interception / execução real** em `src/hooks/useUpdateQuotes.js`.
3. **Rota de busca por tipo de ativo** em `src/services/assetService.js`.

```
src/App.jsx
onUpdateQuote={() => handleUpdateAsset(asset, t, onSuccess)}
```

```
src/hooks/useUpdateQuotes.js
const incoming = await fetchAssetData(asset);
const mergedUpdates = {
  price: incoming.price ?? asset.price,
  priceOriginal: incoming.priceOriginal ?? asset.priceOriginal ?? null,
  priceCurrency: incoming.priceCurrency ?? asset.priceCurrency ?? 'BRL',
  dividendYield: preserveExistingValue(incoming.dividendYield, asset.dividendYield),
  ...
};
onSuccess(asset.id, mergedUpdates);
```

```
src/services/assetService.js
if (
  type.includes('eua') ||
  type.includes('internacional') ||
  type.includes('reit') ||
  type.includes('etf internacional') ||
  type.includes('global')
) {
  return fetchAlphaVantageData(ticker, asset.moeda || 'USD');
}
```

É aqui que a requisição da Alpha Vantage realmente acontece:

```
const params = new URLSearchParams({
  function: 'GLOBAL_QUOTE',
  symbol,
  apikey: apiKey
});

const response = await fetch(`https://www.alphavantage.co/query?${params.toString()}`, {
  headers: { Accept: 'application/json' }
});
```

2) Por que a Alpha Vantage continua falhando

A chave da API não está “se perdendo” em um contexto/global invisível. A leitura atual está explícita em `src/services/assetService.js`:

```
const getAlphaVantageApiKey = () => {
  const configuredKey = typeof import.meta !== 'undefined' && import.meta.env
    ? import.meta.env.VITE_ALPHA_VANTAGE_API_KEY
    : '';

  return configuredKey || getStorageKey('goes_compra_certa_alpha_vantage_api_key');
};
```

A chave é salva em settings em `src/pages/UserSettingsPage.jsx`:

```
alphaVantageApiKeyStorage.set(alphaVantageApiKey);
```

O storage usa a chave do projeto:

```
src/constants/config.js
ALPHA_VANTAGE_API_KEY: 'goes_compra_certa_alpha_vantage_api_key'
```

Ponto crítico: a falha real não é “contexto perdido”. Ela acontece quando a API devolve vazio, informa que a chave está inválida, ou não entrega `Global Quote -> 05. price`.

```
const quote = data?['Global Quote'];
const originalPrice = Number(quote?['05. price']);

if (!Number.isFinite(originalPrice) || originalPrice <= 0) {
  console.warn(`[AlphaVantage] Cotação inválida para ${symbol}.`);
  return null;
}
```

Além disso, esse bloco também bloqueia quando a resposta vem com `Note`, `Information` ou `Error Message`:

```
if (data?.Note || data?.Information || data?.['Error Message']) {  
  console.warn('[AlphaVantage] Resposta informativa:', ...);  
  return null;  
}
```

Conclusão: o código não está “quebrando” no meio do caminho; ele está devolvendo `null` porque a resposta da API não atende ao contrato esperado. A UI então só mostra erro genérico, mas o ponto exato do problema é a resposta da Alpha Vantage / ausência de chave válida.

3) Como o cálculo de `valorInvestido` e `valorTotal` está sendo feito

O cálculo atual está em `src/services/calculationService.js`:

```
const quantity = Number(item.qty) || 0;  
const currentUnitPrice = Number(item.price) || 0;  
const averageUnitPrice = Number(item.pm) || 0;  
const valorTotal = quantity * currentUnitPrice;  
const valorInvestido = isCrypto(item.type, item.ticker)  
  ? averageUnitPrice  
  : quantity * averageUnitPrice;  
const lucro = valorTotal - valorInvestido;
```

Para um ativo com quantidade = 2, a regra exata é:

```
valorTotal = 2 * item.price  
valorInvestido = 2 * item.pm // para ativos não cripto  
lucro = valorTotal - valorInvestido
```

Exemplo numérico:

```
item.qty = 2  
item.pm = 100  
item.price = 120  
  
valorTotal = 2 * 120 = 240  
valorInvestido = 2 * 100 = 200  
lucro = 240 - 200 = 40
```

Ou seja, o sistema calcula o custo e o valor atual multiplicando pela quantidade, e não por “um único preço” sem quantidade.

4) De onde vem o valor de pm no banco de dados

O `pm` vem diretamente do array de ativos lido do `localStorage`, em `src/hooks/useAssets.js` e `src/services/storageService.js`:

```
src/hooks/useAssets.js
const [assets, setAssets] = useState(() => assetsStorage.get());
```

```
src/services/storageService.js
const saved = localStorage.getItem(STORAGE_KEYS.ASSETS);
return saved ? JSON.parse(saved) : [];
```

```
src/constants/config.js
ASSETS: 'goes_compra_certa_assets'
```

Portanto, o valor de `item.pm` que o cálculo usa é exatamente o valor persistido em:

```
localStorage['goes_compra_certa_assets'] -> asset.pm
```

Se esse campo estiver corrompido como total em vez de unitário, o cálculo fica distorcido exatamente como você está vendo na tela.

5) Ponto exato do bug ancorado

- **Atualização de cotação EUA:** `src/hooks/useUpdateQuotes.js` + `src/services/assetService.js`
- **Falha da Alpha Vantage:** a resposta da API não atende ao contrato esperado; código retorna `null` e a UI só exibe erro genérico
- **Cálculo de quantidade e pm:** `src/services/calculationService.js` usa o `pm` diretamente salvo em `localStorage`

Em resumo: não há “outra lógica secreta”; o bug está ancorado no ponto exato em que:

1. a API de cotações dos EUA retorna payload inválido;
2. o código trata isso como falha e interrompe a atualização;
3. o cálculo de lucros usa o `pm` persistido no armazenamento do app, sem correção defensiva de dados corrompidos.